

Lab 1: ROM as Control Store

Microprocessors (010153521) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** A microprocessor with no instruction register and no opcode decode – the ROM word *is* the control word

Objectives

- Explain the difference between a *control-word* machine (this lab, where every ROM bit *is* a control signal directly – there is no opcode, no Instruction Register, and no decode step at all) and a *hardwired FSM* control unit (Lab 2’s EC-1, which decodes an opcode into signals).
- Build a self-addressing program counter: a register that feeds its own next value back from either an incrementer or a ROM-supplied jump address.
- Design the small piece of glue logic that sits *between* the ROM and the PC and decides, every cycle, whether to jump.
- Hand-assemble a control-word microprogram, load it into ROM, and verify a 0-to-10 counting sequence cycle by cycle.

Quick Reference: The Control Word

Program memory is 16×8 -bit ROM. Unlike EC-1, there is **no opcode and no operand split** – every one of the 8 bits drives something directly:

bit 7 ... bit 4		bit 3 ... bit 0	
SEQ	LD_A CLR_A OUT	ADDR[3:0]	

LD_A, CLR_A, OUT drive the Accumulator side directly. ADDR[3:0] is a jump target – meaningful only when SEQ=0. SEQ=1 means “ignore ADDR, just advance to PC+1.” There is one piece of logic *outside* the ROM, sitting between it and the PC:

$$PC_load = \overline{SEQ} \cdot (OUT + NEQ_10)$$

where NEQ_10 is a comparator flag (= 1 when ACC $\neq$ 10). When PC_load=1, PC $\leftarrow$ ADDR; otherwise PC $\leftarrow$ PC+1. This one equation is the *entire* control unit – there is nothing else.

Quick Reference: Control-Word “ISA”

This machine has no fixed opcode field, so there is no fixed instruction set in the EC-1 sense – any of the four control bits can be asserted in any combination in a single ROM word (this is *horizontal microcode*). The mnemonics below name the single-bit-at-a-time words this lab’s programs actually use; multiple bits can still be combined in one word (Exercise 5’s program does exactly that at address 5, combining JMP with OUT).

Operation	Mnemonic	Encoding (SEQ LD_A CLR_A OUT ADDR)	Effect
Clear ACC	CLA	1 0 1 0 0000	$ACC \leftarrow 0$
Output ACC	OUTA	1 0 0 1 0000	$Output \leftarrow ACC$
Increment ACC	INC	1 1 0 0 0000	$ACC \leftarrow ACC + 1$
No-op	NOP	1 0 0 0 0000	advance to PC+1 only
Jump if $ACC \neq 10$	JNZ10 addr	0 0 0 0 aaaa	if $ACC \neq 10$: $PC \leftarrow addr$; else $PC \leftarrow PC+1$
Unconditional jump	JMP addr	0 0 0 1 aaaa	$PC \leftarrow addr$ regardless of ACC; also outputs ACC

Note JMP's encoding is JNZ10's with OUT=1: from the PC_load equation, OUT=1 forces the jump unconditionally, since $OUT + NEQ_{10} = 1$ no matter what ACC holds.

Datapath

Figure 1 is the actual Logisim-Evolution circuit you will build – build *exactly this*, component for component, using the same names.

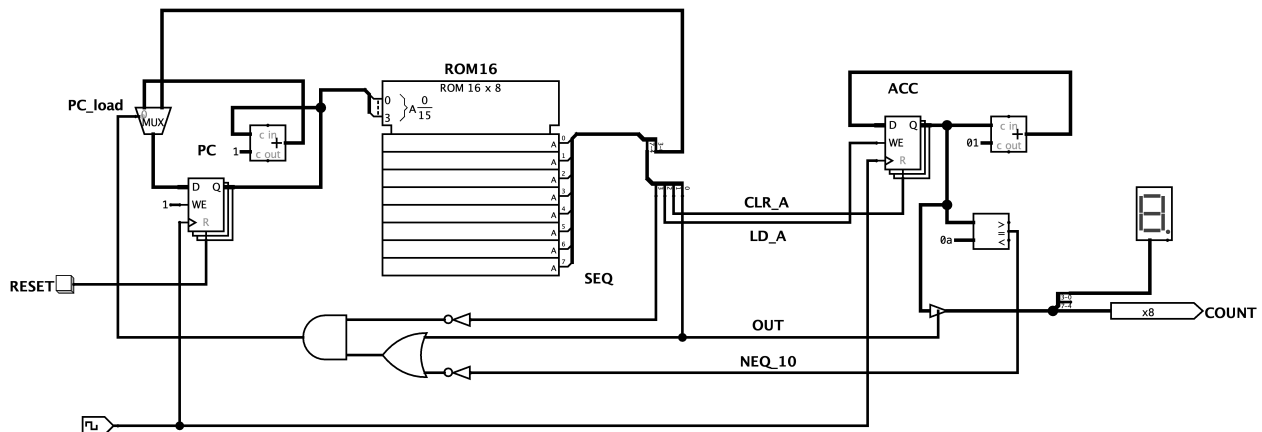


Figure 1: The reference circuit. Left: PC_load selects the MUX feeding PC's D input, either from the +1 adder (constant 1) or looped back from ROM16's own low nibble; PC addresses ROM16 directly (no MAR). Centre: ROM16 (16×8) splits its output into SEQ, CLR_A, LD_A, OUT and the 4-bit jump address fed back to the MUX. Right: ACC is a register with its own +1 adder (constant 01) feeding back into its D input, a comparator against constant 0a (hex 0x0A = 10) producing NEQ_10, and a tri-state buffer gated by OUT driving the 8-bit COUNT bus and a 7-segment display. Bottom: the AND/OR glue logic computing PC_load, plus RESET and the Clock.

The Microprogram: Count 0 to 10

Addr	Meaning	SEQ	LD_A	CLR_A	OUT	ADDR	Hex
0	Clear ACC	1	0	1	0	0000	0xA0
1	Output ACC	1	0	0	1	0000	0x90
2	Increment ACC	1	1	0	0	0000	0xC0
3	Wait (no-op)	1	0	0	0	0000	0x80
4	Jump to 1 if $ACC \neq 10$	0	0	0	0	0001	0x01
5	Jump to self, keep outputting	0	0	0	1	0101	0x15

Before you build anything: using the PC_load equation from the summary box, trace what happens at address 4 (SEQ=0) when $ACC \neq 10$, and separately when $ACC = 10$. Then trace address 5:

why, once reached, does the machine loop on itself *forever* regardless of NEQ_10?

Quick Experiments (Try It)

Try It 1: PC jump mux, in isolation

Build just the left portion of Figure 1 (PC, its +1 adder, the MUX – feed ADDR and PC_load from switches, not yet from ROM).

1. With PC_load=0, does PC count up on every clock edge?
2. Poke ADDR=0101 and set PC_load=1. What is PC after the next clock edge? What if you leave PC_load=1 and keep clocking?

Try It 2: Accumulator + comparator, in isolation

Build just the right-hand portion of Figure 1 (ACC, its +1 adder, the comparator against 0a, and the tri-state buffer – feed LD_A, CLR_A, OUT from switches).

1. Clear ACC, then clock LD_A=1 ten times. Does NEQ_10 go low exactly when ACC reaches 1010?
2. With OUT=0, does the COUNT bus show anything? What does Logisim display for a tri-state buffer's output when disabled?

Try It 3: The glue logic alone

Build $PC_load = \overline{SEQ} \cdot (OUT + NEQ_10)$ from an AND gate and an OR gate, driven by three switches (SEQ, OUT, NEQ_10). (The reference circuit doesn't use a separate NOT gate at all – it negates the SEQ input pin directly on the AND gate. Try building it that way too.)

1. Fill in all 8 rows of its truth table by hand, then check every row against your circuit.
2. Which single input, if you don't know the other two, tells you the most about whether PC_load will be 1?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: PC side (25 min)

Build the left portion of Figure 1 in full: the 4-bit PC register (WE tied to constant 1, so it latches every clock edge; R from RESET), its +1 adder, and the MUX. Wire PC's output directly to where ROM16's address will go.

In-Lab Exercise 2: Accumulator side (30 min)

Build ACC: the 4-bit register (WE = LD_A, R = CLR_A), its own +1 adder feeding back into its D input, a tri-state output buffer, and a comparator against constant 0a (0x0A = 10) producing NEQ_10.

In-Lab Exercise 3: ROM16 and the control word (20 min)

Add a 16×8 -bit **ROM** (ROM16), addressed by PC. Split its 8-bit output into SEQ, CLR_A, LD_A, OUT (1 bit each) and the 4-bit jump address using a Splitter. Wire the three ACC control bits straight to Exercise 2's circuit, and the jump address straight to Exercise 1's MUX input 1.

In-Lab Exercise 4: The glue logic (15 min)

Build $PC_load = \overline{SEQ} \cdot (OUT + NEQ_10)$ from Try It 3, and wire it to the MUX's select input. This is the *entire* control unit for this machine – notice there is no state register anywhere in it.

In-Lab Exercise 5: Assemble, load, and run (30 min)

Wire Exercises 1–4 together exactly as in Figure 1, with a Button for RESET, a Clock, and a HexDisplay on COUNT. Hand-assemble the microprogram from the table above and load it into ROM16 with *Load Image*:

```
v2.0 raw
a0 90 c0 80 01 15 00 00 00 00 00 00 00 00 00
```

Single-step the clock and confirm: COUNT shows 0, 1, 2, ..., 10 in turn, one value per pass through address 1, and then the machine settles at address 5, continuously re-displaying 10 forever. ☐ Count sequence matches. ☐ Machine settles at address 5 (watch PC stop advancing past 5).

AI Collaboration

Try these prompts with an AI tool:

- “What is the architectural difference between a machine where the ROM word is decoded by an FSM, versus one where the ROM word directly drives the datapath?”
- “Given $PC_load = \text{NOT}(SEQ) \text{ AND } (OUT \text{ OR } NEQ_10)$, is there a smaller circuit that computes the same function?”
- “What Logisim-Evolution component implements a 4-bit magnitude comparator, and how do I wire one input to a fixed constant?”

Critical check – verify, don’t just accept: Ask the AI whether this machine’s control unit qualifies as a “Moore machine,” a “Mealy machine,” both, or neither. Then justify the answer yourself, in writing, from the actual circuit – PC_load depends combinationally on NEQ_10 , which itself depends on the *current* value of a register (ACC), not on any external input. Does that change the classification the AI gave you?

Know the limit: AI tools often default to describing microprocessors in terms of fetch/decode/execute because that is the overwhelmingly common case in their training data. This machine has no decode stage at all – if you ask a leading question (“how does this machine decode its opcode?”), many models will happily invent a decode step that doesn’t exist here rather than telling you the premise is wrong.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Six clock cycles from reset

Assume PC resets to 0. Fill in a table with columns *Cycle*, *PC*, *Control word (hex)*, *ACC*, *COUNT* for the first six clock edges. At which cycle does COUNT first show a nonzero value?

Find the Bug 1: A swapped mux input

A student wired ROM16's jump-address field into the MUX's input 0 and the +1 adder into input 1 (i.e. the two mux inputs from Figure 1 are swapped), but the PC_load equation is otherwise correct and unchanged. Predict exactly what PC does starting from address 0, for at least the first 8 cycles. Does the machine still count correctly?

Write Code on Paper 1: Design a count-down microprogram

Using the same control-word format, write a new 16-word microprogram that counts *down* from 10 to 0 (instead of up), still outputting every value and settling into a self-loop at 0. You will need a different comparator target (NEQ_0, i.e. $ACC \neq 0$) and a decremter instead of an incrementer. Write out the full table (address, mnemonic meaning, all five control fields, hex) as in the Quick Reference above.

Draw on Paper 1: Control-word datapath, from memory

Without looking at Figure 1, draw the complete datapath from memory: PC, its +1 adder, the MUX, ROM16, the Splitter's five output fields, ACC, its own +1 adder, the tri-state buffer, the comparator, and the glue logic feeding the MUX's select line. Compare against the real figure afterward and mark anything you got wrong or omitted.